Double Descent 2: Feature learning and Frozen kernel Model-Wise Double Descents

Viktor Veselý *v.vesely@student.rug.nl*
University of Groningen

Abstract

Double descent describes the non-monotonic generalization behavior of neural networks as model capacity or training time increases. In this work, we identify two distinct mechanisms underlying model-wise double descent: feature-learning double descent (FLDD) and frozen-kernel double descent (FKDD). We show that FLDD arises when neural networks learn features that accelerate the fitting of noisy training labels, leading to a transient deterioration in generalization. In contrast, FKDD emerges from the slow fitting of residual errors along small-eigenvalue modes of the empirical neural tangent kernel, which transfer poorly to unseen data. We distinguish these mechanisms through controlled comparisons between feature-learning and frozen-kernel dynamics and introduce a sample-level measure of how feature learning facilitates noise fitting. Through targeted feature-learning ablations, we identify training samples responsible for forming noise-fitting features and demonstrate that suppressing their contribution to feature learning reduces the early double-descent peak without preventing the model from fitting those samples.

1 Introduction

1.1 Contributions

Focusing on MNIST and CIFAR10.

- **(C1) There are two double descents** We identify for each deep architecture and a task there are two double desents. One that happens sooner (epochs and widths) due to feature learning accelerating the learning of a noise. Second happening much later (with epochs) that happens much later due to slow learning modes (define by the smallest non-zero eigenvalue of the empirical ntk) producing large parameters and transfering badly on validaiton.
- **(C2) Identification of feature learning double descent mechanism** We propose and test a mechanism for the rise of the feature learning double descent to be a formation of features which aid with learning noise. We identify samples that cause these features to form and disable feature learning for them (but the model is still being able to fit them) we observe a significant drop on the double desent peak.

2 Background

We consider a supervised learning setting with a training dataset $\mathbb{S} = \{\boldsymbol{x}_s, \boldsymbol{y}_s\}_{l=1}^{S}$ of S i.i.d. samples, where $\boldsymbol{x} \in \mathbb{R}^{d_x}$ denotes the input and $\boldsymbol{y} \in \mathbb{R}^{d_y}$ the corresponding real valued output for regression and $\boldsymbol{y} \in \{0,1\}^{d_y}$ the one-hot vector over d_y classes for classification. Analogously, $\mathbb{Q} = \{\boldsymbol{x}_q, \boldsymbol{y}_q\}_{q=1}^{Q}$ defines the testing dataset with Q samples.

We define $\boldsymbol{\theta}_{n,p} \in \mathbb{R}^p$ to be p parameters of a neural network $f_{\boldsymbol{\theta}}$ trained for n epochs. We train the parameters by minimizing the training loss $\mathcal{L}_{\mathbb{S}}$ which is instantiated to MSE for regression tasks and CE for classification tasks. Test loss $\mathcal{L}_{\mathbb{Q}}$ is used to measure the final performance.

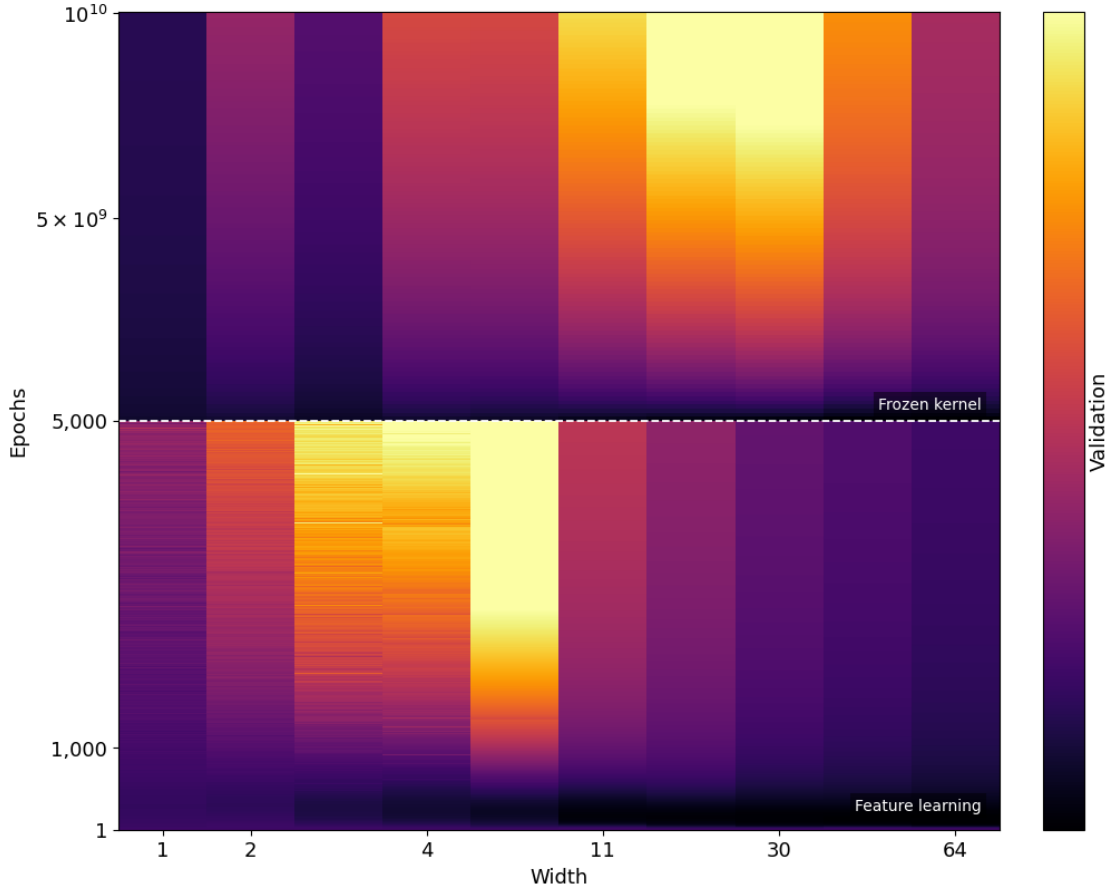


Figure 1: Showcase of two distinct model-wise double descents happening at a different place for the same task, architecture, data ordering, learning rates, amount of learnable parameters. The lower part of the plot shows the early Feature Learning double descent, the upper part of the plot shows the later frozen kernel double descent. Two distinct mechanims give rise to each, see the text for full explanation.

We train our models with the plain stochastic gradient descent for random batches $\mathbb{B} \subset \mathbb{S}$ and a constant learning rate η, yielding the update rule $\theta_{t+1} = \theta t + \eta \nabla_{\theta_t} \mathcal{L}_{\mathbb{B}}(\theta_t)$.

To induce double descent in classification tasks, we introduce label noise at a rate $\eta \in [0, 1]$. Before training, we select a fraction η of the training samples uniformly at random and replace each selected label with a label drawn uniformly from the remaining $d_y - 1$ classes for classification or drawn uniformly from other samples otputs for regression. Both the selected samples and their corrupted labels remain fixed throughout training, while the test labels remain unchanged.

2.1 Taylor approximation of the SGD learning dynamics

Definitions We follow the standard calculus notation for stochastic gradient descent on a training batch $\mathbb{B} \subset \mathbb{S}$

$$\boldsymbol{b} \coloneqq (f_{\boldsymbol{\theta}}(\boldsymbol{x}_0) : \ldots : f_{\boldsymbol{\theta}}(\boldsymbol{x}_{|\mathbb{B}|})), \quad \boldsymbol{x}_i \in \mathbb{B} \quad \text{Batch outputs' concatenation;}$$
$$\boldsymbol{g}_{\mathbb{B}} \coloneqq \nabla_{\boldsymbol{b}} \mathcal{L}_{\mathbb{B}} \quad \text{Output training gradient;}$$
$$\boldsymbol{J}_{\mathbb{B}} \coloneqq \nabla_{\theta} \boldsymbol{b} \quad \text{Jacobian of batch otuputs;}$$
$$\boldsymbol{p}_{\mathbb{B}} \coloneqq \boldsymbol{J}_{\mathbb{B}}^{\top} \boldsymbol{g}_{\mathbb{B}} \quad \text{Parameter training gradient for the batch.}$$
$$K_{\mathbb{X}\mathbb{Y}} \coloneqq \boldsymbol{J}_{\mathbb{X}} \boldsymbol{J}_{\mathbb{Y}}^{\top} \quad \text{Cross-kernel matrix between set } \mathbb{X} \text{ and set } \mathbb{Y}$$

Feature learning identifications We work with the following second taylor approximation of the traing loss change $\Delta\mathcal{L}_{\mathbb{S}}$ for the actual SGD step

$$\Delta\mathcal{L}_{\mathbb{S}} = \sum_{\mathbb{B}\subset\mathbb{S}} \Delta\mathcal{L}_{\mathbb{B}} \tag{1}$$

$$\Delta\mathcal{L}_{\mathbb{B}} \approx \Delta t \dot{\mathcal{L}}_{\mathbb{B}} + \frac{(\Delta t)^2}{2} \ddot{\mathcal{L}}_{\mathbb{B}} \tag{2}$$

$$\dot{\mathcal{L}}_{\mathbb{B}} = -\boldsymbol{g}_{\mathbb{B}}^{\top} \boldsymbol{J}_{\mathbb{B}} \boldsymbol{J}_{\mathbb{B}}^{\top} \boldsymbol{g}_{\mathbb{B}} \tag{3}$$

$$\ddot{\mathcal{L}}_{\mathbb{B}} = \underbrace{\boldsymbol{p}_{\mathbb{B}}^{\top} \boldsymbol{J}_{\mathbb{B}}^{\top} (\nabla_f^2 \mathcal{L}_{\mathbb{S}}) \boldsymbol{p}_{\mathbb{B}}}_{\text{Loss curvature}} + \underbrace{\boldsymbol{p}_{\mathbb{B}}^{\top} \left(\sum_{i}^{d_x} (\boldsymbol{g}_{\mathbb{B}})_i (\nabla_{\boldsymbol{\theta}}^2 f)_i \right) \boldsymbol{p}_{\mathbb{B}}}_{\text{Feature-learning curvature}} \tag{4}$$

with the time step being equal to the learning rate $\Delta t = \eta$. However, later we show that the approximation in Equation 2 is not numerically sufficient due to the loss changing non-linearly across the fixed path of the SGD update, more on that in Appendix A.3. The derivation of the terms is included in Appendix A.1. Second order taylor approximation is the minimal order sufficient to explicitly identify the feature learning effect. Notice that the feature leanring term is precisely 0 with the frozen kernel models which gives us a way to naturally define what feature learning is.

We follow a loose definition of feature learning, where the current task and architecture pair cause the Jacobian to substantially diverge from its initial value $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_0)$, the feature learning term can then be used to capture the instantenous effect of the change in the jacobian on the training/test loss.

Transfer on the validation Equations for the first order transfer metric definition per sample

2.2 Kernel regression

Formalize the kernel regression quickly here and then just reuse it in the later sections

3 Methodology

3.1 Identification of two Double Descent mechanisms

Showing the distinction We can identify the two distinct double double descents by starting from a common point: with the same architecture, task, data ordering, initial features, weight initializations, learning rate, epoch budget T etc. Then we continue with an feature learning ablation, we train two copies of the same starting point one with feature feature learning where the parameters fully free to move and the second copy is with frozen kernel, where the initial features stay the same. The cleanest way of freezing the kernel for a function $f_{\boldsymbol{\theta}_t}$ is to introduce a second parameter vector $\boldsymbol{w} \in \mathbb{R}^{|\boldsymbol{\theta}|}$ which learns the tangent features:

$$\bar{f}_{\boldsymbol{\theta}_t}(\boldsymbol{x}; \boldsymbol{w}) = f_{\boldsymbol{\theta}_t}(\boldsymbol{x}) + (\nabla_{\boldsymbol{\theta}_t} f_{\boldsymbol{\theta}_t}) \boldsymbol{w} \tag{5}$$

where $\boldsymbol{\theta}_t$ will be frozen and $\boldsymbol{w}$ will be updated freely. Substituting $\boldsymbol{\theta}_0$ and learning showcases that the double descent dissapears for the frozen kernel regime (show a figure here).

To

Showing the coexistence The coexistence of the two DDs for the same full feature learning run is this: - for the tasks and architectures presented here it is computationally feasible to empirically observe the FKDD in the actual feature learning run since as seen from the figure the occurance of FKDD happens in very distant epochs (around $5 \cdot 10^9$) - thus we based the coexistence of the two DDs based on the assumption that the movement of the kernel (feature learning) slows down as the training progresses (which is not theoratically guaranteed but it is a common empirical behavior (**?**)) - Using this assumption we can freeze the actual learned model $\boldsymbol{\theta}_T$ by again using Equation 5 and using kernel regression to display the validation error given the frozen kernel assumption

3.2 Feature learning Double Descent Mechanism

In frozen kernel regime the peak is caused by the following conditions: the empirical ntk contains small non-zero eigenvalues AND that eigmode has some residual power left (gradient wants to fit it) AND that eigenmode transfers harmfully on the validation. The intuitve mechanism is then that in order to fit the small eigenmodes it requires large coefficients, which transfer very poorly on the validation (since they are taylord for the noise). The frozen kernel regime operates in the eigenmodes, our hypothesis about the FL DD operates from the samples' perspective. We call this double descent FKDD

We hypothesize that the Fl DD is caused by some training samples constructing features which make learning noise samples easier, conditions: i) there will be samples which create features which are beneficial for the noise AND ii) the network has sufficient capacity to fit the noise (and it will be fitted quickly) AND iii) the noise samples will transfer VERY poorly to the validation. We call this double descent FLDD.

Now formulating i) ii) and iii) exactly

i) comes from the following property. Let $\mathbb{L} \subset \mathbb{B}$ be the noise samples in the current training batch. We can measure how easily can the noise be learned under the current features/jacobian lets call this property $l(\boldsymbol{\theta})$:

$$l(\boldsymbol{\theta}) = \boldsymbol{g}_{\mathbb{L}}^{\top} J_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}} \tag{6}$$

$$\dot{l}(\boldsymbol{\theta}) = \underbrace{\dot{\boldsymbol{g}}_{\mathbb{L}}^{\top} J_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}}_{\text{Changes in the loss}} + \underbrace{\boldsymbol{g}_{\mathbb{L}}^{\top} \dot{J}_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}}_{\text{Changes in the features}} + \underbrace{\boldsymbol{g}_{\mathbb{L}}^{\top} J_{\mathbb{L}} \dot{\boldsymbol{p}}_{\mathbb{B}}}_{\text{SGD does not change the direction in one step thus 0}} \tag{7}$$

we are only interested how much does the feature learning influence the ease of noise fitting (the middle term lets call it h_i) .

We are not claiming that these are the sufficient conditions for the FLDD to arise, we acknowledge its depedence also on other factors such as noise percentage, optimizers, batch size, and learning rate (citations here). We merely identify the feature learning contributon of the DD peak. This mechanism can be validated using the following ablation: -

4 Experiments

References

A Appendix

A.1 Derivation of the 2nd order taylor loss changes

$$\mathcal{L}_{\mathbb{Q}}(t) = \mathcal{L}_{\mathbb{Q}}(\theta(t)) \qquad \text{Validation loss} \tag{8}$$

$$\theta_t(\tau) = \theta(t) - \tau \nabla_\theta \mathcal{L}_S(\theta(t)) \qquad \text{Linear interpolation of one SGD step} \tag{9}$$

$$\mathcal{L}_{\mathbb{Q}}(\theta_t(\Delta t)) = \mathcal{L}_{\mathbb{Q}}(\theta_t(0)) + \int_0^{\Delta t} \dot{\mathcal{L}}_{\mathbb{Q}}(\theta_t(\tau))\, d\tau \qquad \text{Approximate with Taylor series} \tag{10}$$

$$\mathcal{L}_{\mathbb{Q}}(\theta_t(\Delta t)) \approx \mathcal{L}_{\mathbb{Q}}(\theta_t(0)) + \Delta t\, \dot{\mathcal{L}}_{\mathbb{Q}}\big|_{\tau=0} + \frac{1}{2}\Delta t^2\, \ddot{\mathcal{L}}_{\mathbb{Q}}\big|_{\tau=0} \qquad \text{Approximated at } \tau = 0 \tag{11}$$

$$\dot{\mathcal{L}}_{\mathbb{Q}} = -g_Q^\top J_Q J_S^\top g_S \qquad \text{First-order validation change} \tag{12}$$

$$\dot{\mathcal{L}}_{\mathbb{Q}} = -\sum_i \underbrace{\|g_S^{(i)}\|}_{\text{train_push}} \underbrace{g_Q^\top J_Q (J_S^\top)^{(i)} \hat{g}_S^{(i)}}_{\text{transfer}} \qquad \text{Per-sample push and transfer} \tag{13}$$

$$\dot{\mathcal{L}}_{\mathbb{Q}} = \nabla_\theta \mathcal{L}_{\mathbb{Q}}(\theta)^\top \dot{\theta}_t(\tau) \qquad \text{Chain rule} \tag{14}$$

$$\ddot{\mathcal{L}}_{\mathbb{Q}} = \nabla_\theta \mathcal{L}_{\mathbb{Q}}(\theta)^\top \ddot{\theta}_t(\tau) + \dot{\theta}_t(\tau)^\top \nabla_\theta^2 \mathcal{L}_{\mathbb{Q}}(\theta) \dot{\theta}_t(\tau) \qquad \text{Product rule} \tag{15}$$

$$\text{One gradient step of SGD is linear, thus } \ddot{\theta}_t(\tau) = \vec{0} \tag{16}$$

$$\ddot{\mathcal{L}}_{\mathbb{Q}} = \dot{\theta}_t(\tau)^\top \nabla_\theta^2 \mathcal{L}_{\mathbb{Q}}(\theta) \dot{\theta}_t(\tau) \qquad \text{Second-order validation change} \tag{17}$$

$$\nabla_\theta^2 \mathcal{L}_{\mathbb{Q}} = \nabla_\theta \left((\nabla_\theta f)^\top \nabla_f \mathcal{L}_{\mathbb{Q}} \right) \qquad \text{Expand the validation Hessian} \tag{18}$$

$$\nabla_\theta^2 \mathcal{L}_{\mathbb{Q}} = (\nabla_\theta f)^\top \nabla_f^2 \mathcal{L}_{\mathbb{Q}} (\nabla_\theta f) + \sum_a (\nabla_f \mathcal{L}_{\mathbb{Q}})_a (\nabla_\theta^2 f)_a \qquad \text{Second-order chain rule} \tag{19}$$

$$\begin{aligned} \ddot{\mathcal{L}}_{\mathbb{Q}} = {} & (\nabla_\theta \mathcal{L}_S)^\top \left((\nabla_\theta f)^\top \nabla_f^2 \mathcal{L}_{\mathbb{Q}} (\nabla_\theta f) \right) (\nabla_\theta \mathcal{L}_S) \\ & + (\nabla_\theta \mathcal{L}_S)^\top \left(\sum_a (\nabla_f \mathcal{L}_{\mathbb{Q}})_a (\nabla_\theta^2 f)_a \right) (\nabla_\theta \mathcal{L}_S) \end{aligned} \qquad \text{Fixed-feature + feature-learning curvature} \tag{20}$$

$$\underbrace{(\nabla_\theta \mathcal{L}_S)^\top \left((\nabla_\theta f)^\top \nabla_f^2 \mathcal{L}_{\mathbb{Q}} (\nabla_\theta f) \right) (\nabla_\theta \mathcal{L}_S)}_{\text{fixed-feature / loss curvature}}$$

$$+ \underbrace{(\nabla_\theta \mathcal{L}_S)^\top \left(\sum_a (\nabla_f \mathcal{L}_{\mathbb{Q}})_a (\nabla_\theta^2 f)_a \right) (\nabla_\theta \mathcal{L}_S)}_{\substack{\text{feature-learning curvature} \\ \approx 0 \text{ for linear / infinite-width frozen-feature models}}}$$

A.2 Hessian and Jacobian free formulation

A.3 Multistep integrated taylor approximation

A.4 Calculation of cumulative test displacement Δq